ML concepts

- $\underbrace{p(\boldsymbol{\theta}|\mathcal{D})}_{\textbf{posterior}} \propto \underbrace{p(\mathcal{D}|\boldsymbol{\theta})}_{\textbf{likelihood}} \underbrace{p(\boldsymbol{\theta})}_{\textbf{prior}}$
- ℓ_0 norm: # non-zero elements
  ℓ_1 norm: $\sum_{i=1}^{k} |w_i|$
  ℓ_2 norm: $\sqrt{\sum_{i=1}^{k} (w_i)^2}$
- **Training data**: data that the model will be trained on
  Test data: real-world data not accessible during model training
  Validation data: subset of training data hold out from training in order to gauge performance on test data to better tune hyperparameter (regularization constant, learning rate, # hidden units, etc.)
- **Generalization**: how well the model performs on real-world data
  Overfitting: poor generalization $\Rightarrow$ solution: validation set, more training data, reduce model complexity, fit to many different degrees, penalize on weights
- **Discriminative**: a class of model type that models $p(Y|X)$. ML models of this class find the decision boundary directly.
  Generative: a class of model type that is modeling $p(X|Y)$ (generating data). ML models of this class model distributions and choose the one with the highest probability.
- **Parametric models** have a fixed number of parameters (need few data, fast, stronger Strong distributional assumptions): Linear/Logistic Regression, Naïve Bayes, Discriminant Analysis, Neural Networks
  Non-parametric models have the number of parameters grow with the amount of training data (more flexible, computationally intractable for large datasets): KNN, Decision Tree
- **Predictive / supervised learning**: the goal is to learn a mapping from inputs $\mathbf{x}$ to output y, given a labeled set of input-output pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$. **Descriptive / unsupervised learning**: given input $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^{N}$, and the goal is to find "interesting patterns" in the data $\Rightarrow$ **knowledge discovery**.

Maximum likelihood estimation (MLE)

- Assume each data point is **iid**,

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{N} p(\mathcal{D}|\boldsymbol{\theta})$$
$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p(\mathcal{D}|\boldsymbol{\theta})$$
$$\hat{\boldsymbol{\theta}}^{MLE} = \arg\max_{\boldsymbol{\theta}} p(\mathcal{D}|\boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \log p(\mathcal{D}|\boldsymbol{\theta})$$

Maximum a posteriori (MAP)

- include a prior distribution:

$$L(\boldsymbol{\theta}) = p(\boldsymbol{\theta}) \prod_{i=1}^{N} p(x_i|\boldsymbol{\theta})$$
$$\hat{\boldsymbol{\theta}}^{MAP} = \arg\max_{\boldsymbol{\theta}} p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} [\log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta})]$$

- as we get more and more data, the MAP estimate converges towards the MLE since the **data overwhelms the prior**.

	MLE	MAP
Discriminative (more data; less assumption)	$p(\mathcal{D}\|\boldsymbol{\theta})$ $p(y\|\mathbf{X}, \boldsymbol{\theta})$ linear/logistic regression with polynomial features	$p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta})$ $p(y\|\mathbf{X}, \boldsymbol{\theta})p(\boldsymbol{\theta})$ linear regression with ℓ_2 regularization logistic regression with Laplace prior
Generative (less data; more model assumption)	$p(\mathcal{D}\|\boldsymbol{\theta})$ $p(\mathbf{X}\|y, \boldsymbol{\theta})p(y\|\boldsymbol{\theta})$ naïve Bayes	$p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta})$ $p(\mathbf{X}\|y, \boldsymbol{\theta})p(y\|\boldsymbol{\theta})p(\boldsymbol{\theta})$ naïve Bayes with Laplace smoothing

Linear regression

- A model of the form: $p(y|\mathbf{x},\theta) = \mathcal{N}(y|\mathbf{w}^T\mathbf{x},\sigma^2) \Rightarrow$

$$\hat{y} = \mathbf{w}^T\mathbf{x} + \epsilon$$

  Basis function expansion: $p(y|\mathbf{x},\theta) = \mathcal{N}(y|\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}),\sigma^2)$ where $\boldsymbol{\phi}(\mathbf{x}) = [1, x, x^2, \ldots, x^d]$.

- Applying MLE to linear regression:

$$\begin{aligned}\ell(\boldsymbol{\theta}) = \ell(\mathbf{w},\sigma^2) &= \sum_{i=1}^{N} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{1}{2\sigma^2}(y_i - \mathbf{w}^T\mathbf{x}_i)^2\right)\right] \\ &= -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\underbrace{\sum_{i=1}^{N}(y_i - \mathbf{w}^T\mathbf{x}_i)^2}_{\text{residual sum of squares}}\end{aligned}$$

  Mean squared error (MSE) $= \frac{RSS}{N} = \frac{1}{N}||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2$

$$J(\mathbf{w}) = \frac{1}{2}||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2 \Rightarrow \boxed{\hat{\mathbf{w}}_{OLS} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}}$$

Logistic regression

- **sigmoid function**: $\sigma(a) = \frac{1}{1+\exp(-a)}$, $\frac{d\sigma}{da} = \sigma(1-\sigma)$.
  Logistic function is related to the inverse of **logit function**:

$$\begin{aligned} z &= logit(p) = \log\frac{p}{1-p} \\ p &= logit^{-1}(z) = \frac{1}{1+e^{-z}}\end{aligned}$$

- Binary classification model:

$$p(y|\mathbf{x},\mathbf{w}) = \text{Bern}(y|\sigma(\mathbf{w}^T\mathbf{x}))$$

- Applying MLE to logistic regression gives the **cross-entropy error function**:

$$\begin{aligned} L(\mathbf{w}) &= \prod_{i=1}^{N}\sigma\left(\mathbf{w}^T\mathbf{x}^{(i)}\right)^{\mathbb{I}\left(y^{(i)}=1\right)}\left(1-\sigma\left(\mathbf{w}^T\mathbf{x}^{(i)}\right)\right)^{1-\mathbb{I}\left(y^{(i)}=1\right)} \\ \ell(\mathbf{w}) &= \sum_{i=1}^{N}\left(y^{(i)}\log\sigma\left(\mathbf{w}^T\mathbf{x}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\sigma\left(\mathbf{w}^T\mathbf{x}^{(i)}\right)\right)\right)\end{aligned}$$

  $\frac{\partial\ell}{\partial\mathbf{w}} = \mathbf{X}^T(\boldsymbol{\mu} - \mathbf{y}) = 0$ has no closed-form solution but positive definite Hessian $\Rightarrow$ strictly convex and **gradient descent** can find the unique global minimum:

$$\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k - \underbrace{\eta_k}_{\textbf{learning rate}}\frac{\partial\ell}{\partial\mathbf{w}}$$

- **one-versus-all**:

  1. learn one binary classifier for each class: for each $1 \le k \le K$, treat samples of class k as positive samples and the other as negative samples:

$$p(y=k|\mathbf{x};\mathbf{W},\mathbf{b}) = \frac{1}{1+\exp(-(\mathbf{w}_k^T\mathbf{x} + b_k))}$$

  2. majority vote:

$$\hat{y} = \arg\max_k p(y^{(k)} = 1|\mathbf{x};\mathbf{W},\mathbf{b})$$

- **multi-class logistic regression / maximum entropy classifier**: a model of the form

$$p(y^{(i)} = k|\mathbf{x}^{(i)}, \mathbf{W}) = \frac{\exp\left(\mathbf{w}_k^T \mathbf{x}^{(i)}\right)}{\sum_{c=1}^{k} \exp\left(\mathbf{w}_c^T \mathbf{x}^{(i)}\right)}$$

  Conditional likelihood:

$$L(\mathbf{w}) = \prod_{i=1}^{N} \frac{\exp\left(\mathbf{w}_k^T \mathbf{x}^{(i)}\right)}{\sum_{c=1}^{k} \exp\left(\mathbf{w}_c^T \mathbf{x}^{(i)}\right)}$$

  relationship to logistic regression: $K = 2$

Regularization

- combine original objective with penalty on parameters to avoid overfitting
- **ridge regression / penalized least squares**: linear regression with ℓ_2 penalty on weights

$$J(\mathbf{w}) = \frac{1}{2}\left(||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2 + \lambda||\mathbf{w}||_2^2\right) \Rightarrow \boxed{\hat{w}_{ridge} = \left(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}^T\mathbf{y}}$$

- **lasso (least absolute shrinkage and selection operator) regression**: linear regression with ℓ_1 penalty on weights:
  - Laplace prior on weights and uniform prior on offset term:

$$\mathbf{w} \sim \text{Lap}(\boldsymbol{\mu} = 0, b) \Rightarrow p(\mathbf{w}|b) = \frac{1}{2b}e^{-\frac{|w|}{b}}$$
$$p(\mathbf{w}|\lambda) = \prod_{j=1}^{D} \text{Lap}(w_j|0, 1/\lambda) \propto \prod_{j=1}^{D} e^{-\lambda|w_j|}$$

  - Performing MAP estimation with prior in linear regression:

$$J(\mathbf{w}) = \frac{1}{2}\left(||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2 + \lambda||\mathbf{w}||_1\right)$$

  - leads to sparse weight values: geometrically, as we relax the constraint, the corner of the ℓ_1 ball (corresponding to sparse solutions) is more likely to intersect with the objective ellipsis than the edge
    (ridge regression has no preference for sparsity since the ℓ_2 ball can intersect with the objective at any point)

Naïve Bayes

- Assume the features x_i are **conditionally independent** given the class label y, the **naïve Bayes classifer** (NBC) has the resulting model:

$$p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{i=1}^{N} p(x_i|y = c, \boldsymbol{\theta}_{ic})$$

- MLE for NBC: $y \sim \text{Bern}(\phi), x_m \sim \text{Bern}(\theta_{m,y})$,

$$L(\phi, \boldsymbol{\theta}) = \phi^{N_{y=1}}(1-\phi)^{N_{y=0}} \prod_{m=1}^{M} \theta_{m,1}^{N_{y=1,x_m=1}} (1-\theta_{m,1})^{N_{y=1,x_m=0}} \theta_{m,0}^{N_{y=0,x_m=1}} (1-\theta_{m,0})^{N_{y=0,x_m=0}}$$

- **Laplace smoothing**: λ-smoothing to ensure non-zero probabilities

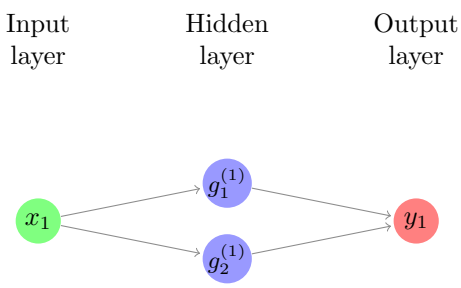
$$\phi_k = \frac{\lambda + \sum_{i=1}^{N} \mathbb{I}_{(x^{(i)}=k)}}{k\lambda + N}$$

- generative models with continuous features: Bernouli class distribution $y \sim \text{Bern}(\phi)$ with Gaussian class-conditional distribution $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y)$.

- **Decision boundaries**:
  - Naïve Bayes assumption: diagonal Σ_y
  - Linear decision boundary: $\Sigma_{y=0} = \Sigma_{y=1}$
  - Quadratic decision boundary: $\Sigma_{y=0} \neq \Sigma_{y=1}$

Neural Networks (MLPs)

- **input layer, hidden layer, output layer**
- **activation function**: functions $g : \mathbb{R}^d \to \mathbb{R}^d$ that take the results of our linear product α^i and applies some (typically non-linear, since otherwise linear layers can be reduced to an exactly equivalent single linear layer) function elementwise:
  - sigmoid function: $g(\alpha^i) = \frac{1}{1+\exp(-\alpha^i)}$
  - **ReLU (rectified linear unit)**: $g(\alpha^i) = \max\{0, \alpha^i\}$
  - identity function
- **Universal Approximation Theorem:** A two layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.
- **backpropagation**:



$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z_1}\frac{\partial z_1}{\partial x} + \frac{\partial y}{\partial z_2}\frac{\partial z_2}{\partial x}$$

- **convolutional neural network (CNN)**: **convolution**:

$$\underbrace{s(t)}_{\textbf{feature map}} = (\underbrace{x}_{\textbf{input}} * \underbrace{w}_{\textbf{kernel}})(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

  Convolution is commutative.
  A typical layer of CNN consists of three stages:

  1. Convolution layer: Affine transform
  2. Detector layer: Nonlinearity
  3. Pooling layer: a **pooling function** replaces the output of the net at a certain location with a summary statistic of the nearby outputs, e.g. **max pooling** operation reports the maximum output within a rectangular neighborhood

K-nearest neighbors (KNN)

- takes a new data point and gathers the labels of the K nearest (determined by some metric) neighbors to determine the label this new point:

$$p(y = c|\mathbf{x}, \mathcal{D}, K) = \frac{1}{K} \sum_{i \in N_K(x,\mathcal{D})} \mathbb{I}(y_i = c)$$

- bias vs variance tradeoff:
  - larger $K \to$ more stable predicted label
  - smaller $K \to$ predicted label more affected by training points
- assumptions:
  - similar points have similar neighbors
  - all feature dimensions are created equally: use neural networks to learn to transform input data into a feature space where distance is more meaningful

Decision trees

- **Classification and regression trees (CART)** are defined by recursively partitioning the input space, and defining a local model in each resulting region of input space
  discrete decision trees can express any function of the input

- Growing a tree:

```
ID3(Examples, Target_attribute, Attributes):
    Create a Root node for the tree
    If all Examples are +, Return Root with label +
    If all Examples are -, Return Root with label -
    If Attributes is empty, Return Root with label = the most common value of
                                                        Target_attribute in Examples
    Otherwise Begin,
        A ← best attribute from Attributes that classifies Examples
        The decision attribute for Root ← A
        For each possible value of, v_i, of A,
            Add a new branch below Root, corresponding to the test A = v_i
            Let Examples_{v_i} be the subset of Examples that have value v_i for A
            If Examples_{v_i} is empty
                Then add a leaf with label = most common value of
                Target_attribute in Examples
                Else add a subtree
                ID3(Examples_{v_i}, Target_attribute, Attributes - {A})
```

- **entropy** quantifies the amount of uncertainty associated with a specific probability distribution
  The higher the entropy, the less confident we are in the outcome:

$$H(Y) = -\sum_{y \in \mathcal{Y}} p(Y = y) \log_2 p(Y = y)$$

  specific conditional entropy:

$$H(Y|X = x) = -\sum_{y \in \mathcal{Y}} p(Y = y|X = x) \log_2 p(Y = y|X = x)$$

  conditional entropy:

$$H(Y|X) = -\sum_{x \in \mathcal{X}} p(X = x) \log_2 p(Y|X = x)$$

  mutual information is a measure of reduced uncertainty about Y by the knowledge of X:

$$I(Y; X) = H(Y) - H(Y|X) = H(X) - H(X|Y)$$

  The best attribute maximizes information gain.

- Handling continuous features: **dyadic decision trees** (split on mid points of features)

- Overfitting problem:
  - Pre-pruning: Fixed depth/number of leaves
  - Post-pruning
  - Complexity penalized model selection:

$$\hat{f} = \arg\min_T \left\{ \frac{1}{n} \sum_{i=1}^{N} \text{loss}\left(\hat{f}_T(X_i), Y_i\right) + \text{pen}(T) \right\}$$

$$\text{loss}\left\{\hat{f}_T(X_i), Y_i\right\} = \left(\hat{f}_T(X_i) - Y_i\right)^2 \qquad \text{regression}$$

$$= \mathbb{I}_{\hat{f}_T(X_i) = Y_i} \qquad \text{classification}$$

Cross validation

- **K-fold cross-validation**:
  - Create K fold partition of the dataset
  - Do K runs: train using $K-1$ partitions and calculate validation error on remaining partition (rotating validation partition on each run).
  - Report average validation error
- **Leave-one-out (LOO) cross-validation**: K-fold with $K = N$
- **Random subsampling**:
  - Randomly subsample a fixed fraction $\alpha N, \alpha \in (0,1)$ of the dataset for validation
  - Compute validation error with remaining data as training data
  - Repeat K times and report average validation error
- Bias-variance tradeoff of choosing K: for large K (close to n)
  - bias of error estimate is small
  - variance of error estimate is large
  - large computation time

  Common choice: $K = 10, \alpha = 0.1$

Kernal regression

- non-parametric regression: **kernel function** $\kappa(\mathbf{x}, \mathbf{x}') \in \mathbb{R}$ (symmetric, non-negative) measures the similarity between objects $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$
- **squared exponential (SE) / Gaussian kernel**:

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2}(\mathbf{x}-\mathbf{x}')^T \mathbf{\Sigma}^{-1}(\mathbf{x}-\mathbf{x}')\right)$$

  If $\mathbf{\Sigma}$ is diagonal,

$$\kappa(\mathbf{x}, \mathbf{x}') \exp\left(-\frac{1}{2}\sum_{j=1}^{D}\frac{1}{\sigma^2}(x_j - x_j')^2\right)$$

  where σ_j is the **characteristic length scale** of dimension j. If $\sigma_j = \infty$, that dimension is ignored (**ARD kernel**)
  If $\mathbf{\Sigma}$ is spherical,

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{||\mathbf{x}-\mathbf{x}'||^2}{2\sigma^2}\right)$$

  where σ^2 is the **bandwidth** (**RBF kernel** since it's only a function of $||\mathbf{x}-\mathbf{x}'||$), overfit when σ is too large

- $$\boldsymbol{\alpha} = (K + \lambda \mathbf{I})^{-1}\mathbf{y}, \text{ where } K_{ij} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

  given a new point $\mathbf{x}$, predict

$$\hat{y} = \sum_{i=1}^{N} \alpha_i k(\mathbf{x}, \mathbf{x}^{(i)})$$

  Remark. $\boldsymbol{\alpha}$ used as a 'normalizer' to allow weighted sum of kernel windows; λ used to ensure the term is invertible

Support vector machine (SVM)

- technique combining the kernel trick plus a modified loss function and producing sparse solution so the prediction only depends on a subset of training data known as **support vectors**
- **maximum margin classifier** formalized in an optimization problem in primal form:

- linear separable case:

$$\arg\min_{\mathbf{w},b} \frac{1}{2}||\mathbf{w}||^2 \text{ s.t. } y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)}+b\right) \geq 1, \forall i$$

- non-linear separable case:

$$\arg\min_{\mathbf{w},b} \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{N}\xi_i \text{ s.t. } \underbrace{\xi_i \geq 0}_{\textbf{slack variables}}, \underbrace{y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)}+b\right) \geq 1-\xi_i}_{\textbf{soft margin constraints}}, \forall i$$

  where C is a regularization parameter that controls the number of errors tolerable on the training set (larger C value means smaller errors and harder margin). Define $C = \frac{1}{(\upsilon N)}$ where $\upsilon \in (0,1]$ controls the fraction of misclassified points allowed during in training $\Rightarrow$ υ-**SVM classifer**

- kernel method:
  - method of lagrange multiplier: for $\min_{\mathbf{x}} f(\mathbf{x})$ s.t. $g(\mathbf{x}) \leq c$, first construct lagrangian

$$\mathcal{L}(\mathbf{x}, \underbrace{\lambda}_{\textbf{dual variable}}) = f(\mathbf{x}) + \lambda(g(\mathbf{x})-c), \text{ then solve } \quad \min_{\mathbf{x}}\max_{\lambda\geq 0}\mathcal{L}(\mathbf{x},\lambda)$$

    by finding saddle point $\nabla\mathcal{L}(\mathbf{x},\lambda)$ s.t. $\lambda \geq 0 \Leftrightarrow$ solving $\nabla f(\mathbf{x}) = \lambda\nabla g(\mathbf{x})$
  - SVM primal to dual: let $f(\mathbf{x}) = \frac{1}{2}||\mathbf{w}||_2^2, g(\mathbf{x}) = y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)}+b\right), c = 1$, then the lagrangian is

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}||\mathbf{w}||_2^2 + \sum_{i=1}^{N}\alpha_i\left[y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)}+b\right)-1\right]$$

    the optimization problem after simplification in dual form is thus:

$$\max_{\alpha}\sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y^{(i)}y^{(j)}\mathbf{x}^{(i)}\mathbf{x}^{(j)} \qquad \text{s.t} \qquad \alpha_i \geq 0, \forall i, \sum_{i=1}^{N}\alpha_i y^{(i)} = 0$$

    which gives

$$\begin{aligned}\mathbf{w} &= \sum_{i=1}^{N}\alpha_i y^{(i)}\mathbf{x}^{(i)}\\ b &= y^{(k)} - \mathbf{w}^T\mathbf{x}^{(k)} \text{ for any } k \text{ where } a_k > 0\\ \hat{y} &= \text{sign}(\mathbf{w}^T\mathbf{x}+b)\end{aligned}$$

    Advantage of dual:
    * reduce the number of parameters (only need to store information of the support vectors)
    * kernel trick by applying $\phi(\mathbf{x})$:

$$\max_{\alpha}\sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y^{(i)}y^{(j)}k(\mathbf{x}^{(i)},\mathbf{x}^{(j)}) \qquad \text{s.t} \qquad \alpha_i \geq 0, \forall i, \sum_{i=1}^{N}\alpha_i y^{(i)} = 0$$

  - **Mercer's Theorem**: valid kernels that correspond to feature vector $\phi(\mathbf{x})$: Mercer kernel K (continuous, symmetric, positive semi-definite)

Principal componenet analysis (PCA)

- We want to find an orthogonal set of K linear basis vectors $\mathbf{v}_j \in \mathbb{R}^M$ and the corresponding scores $\mathbf{z}_i \in \mathbb{R}^K$ such that we minimize the **reconstruction error**

$$J(\mathbf{W},\mathbf{Z}) = \sum_{i=1}^{N}||\mathbf{x}_i - \mathbf{V}\mathbf{z}_i||_2^2$$

  where $\mathbf{V} \in \mathbb{R}^{M\times K}, \mathbf{Z} \in \mathbb{R}^{N\times K}$ and which is equivalent to maximizing the variance of the projected data

$$\sum_{i=1}^{N}(\mathbf{z}_i)^2$$

The optimal solution is obtained by setting $\hat{\mathbf{V}} = \mathbf{V}_L$ that contains the K eigenvectors with largest eigenvalues of the empirical covariance matrix $\hat{\mathbf{\Sigma}} = \frac{1}{N}\mathbf{X}^T\mathbf{X}$ assuming <u>the data are centered</u> (with zero mean; if not, subtract off the sample mean). The optimal low-dimensional encoding of the data is given by $\hat{\mathbf{z}}_i = \mathbf{V}^T\mathbf{x}_i$, which is an orthogonal projection of the data onto the column space spanned by the eigenvectors.
Remarks: any vector $\mathbf{v}$ that maximizes the variance is an eigenvector of $\mathbf{\Sigma}$, i.e. $\mathbf{\Sigma}\mathbf{v} = \lambda\mathbf{v}$ where λ equals the variance of the projections along $\mathbf{v}$.

- Choose K such that we retain some fraction of the variance:

$$\frac{\sum_{i=1}^{K}\lambda_i}{\sum_{i=1}^{d}\lambda_i}$$

- **singular value decomposition (SVD)**:
  - any (real) $N \times D$ matrix $\mathbf{X}$ can be decomposed into

$$\underbrace{\mathbf{X}}_{N\times D} = \underbrace{\mathbf{U}}_{N\times N}\underbrace{\mathbf{S}}_{N\times D}\underbrace{\mathbf{V}^T}_{D\times D}$$

    * $\mathbf{U}$: columns are orthonormal $\Rightarrow \mathbf{U}^T\mathbf{U} = \mathbf{I}_N$, left singular vectors of $\mathbf{X}$ and <u>eigenvectors of $\mathbf{X}\mathbf{X}^T$</u>
    * $\mathbf{S}$: <u>diagonal matrix</u>, diagonal entries $\sigma_k \geq 0$ are singular values of $\mathbf{X} \Rightarrow$ <u>σ_k^2 are eigenvalues of both $\mathbf{X}\mathbf{X}^T$ and $\mathbf{X}^T\mathbf{X}$</u>
    * $\mathbf{V}$: rows and columns are orthonormal $\Rightarrow \mathbf{V}^T\mathbf{V} = \mathbf{V}\mathbf{V}^T = \mathbf{I}_D$, <u>columns are eigenvectors of $\mathbf{X}^T\mathbf{X}$</u>
  - PCA algorithm: given input $\mathbf{X}, \mathbf{X}_{test}, K$,
    1. center data based on training data
    2. $\mathbf{V}_K = \text{eigenvectors}\left(\mathbf{X}^T\mathbf{X}\right)_K$ (top K eigenvectors)
    3. $\mathbf{Z}_{test} = \mathbf{V}_K^T\mathbf{X}_{test}$
  - Kernel PCA algorithm: given input $\mathbf{X}, \mathbf{x}_{test}, L$
    1. kernel matrix $\mathbf{K}_{ij} = k(\mathbf{X}_i, \mathbf{X}_j)$
    2. center K
    3. $\mathbf{U} = \text{eigenvectors}(\mathbf{K})_L, \boldsymbol{\lambda} = \text{eigenvalues}(\mathbf{K})_L$
    4. $\boldsymbol{\alpha}_j = \frac{1}{\sqrt{\lambda_j}}\mathbf{U}_j$
    5. $z_{test,j} = \sum_i \alpha_{ij}k(\mathbf{X}_i, \mathbf{x}_{test})$

<u>*Recommender systems*</u>

- given a matrix $R \in \mathbb{R}^{N\times M}$ with label r_{ij} being rating of user i on item j, our goal is to find $U \in \mathbb{R}^{N\times K}, V \in \mathbb{R}^{M\times K}$ where K is a hyperparameter (higher K gives more accurate predictions and higher complexity):

$$J(U,V) = \min_{U,V} ||R - UV^T||_F^2$$

  since some values of R are not defined, we optimize over the set $S = \{i,j\}$ for all r_{ij} are known:

$$J(U,V) = \min_{U,V} \sum_{i,j\in S}\left(r_{ij} - u^{(i)T}v^{(j)}\right)^2$$

  where $u^{(i)} \in \mathbb{R}^K, v^{(j)} \in \mathbb{R}^K$

- **SGD - method of alternating minimization**:
  1. $\min_U J(U,V) \Rightarrow U^{t+1} \leftarrow U^t - \alpha\nabla_U J$
  2. $\min_V J(U,V) \Rightarrow V^{t+1} \leftarrow V^t - \alpha\nabla_V J$

  add regularization to avoid overfitting:

$$\min_{U,V} J(U,V) = \sum_{i,j\in S}\left[\left(R_{ij} - u^{(i)T}v^{(j)}\right)^2 + \lambda||u^{(i)}||_2^2 + \lambda||v^{(i)}||_2^2\right]$$

Clustering

- goal: automatically partition unlabeled data into groups of similar datapoints for:
  - automatically organizing data
  - understanding hidden structure in data
  - preprocessing for further analysis (visualization)
- **hierarchical clustering** where clusters can be nested inside each other:
  - bottom-up / **agglomerative clustering**: greedy - less accurate but simple to implement
    - start with objects in separate clusters and repeat -
      joins the most similar pair of clusters;
      update the similarity of the new cluster to others until there is only one cluster
    - **single-linkage**: nearest Neighbor: similarity between their closest members
      complete-linkage: furthest neighbors: similarity between their furthest members
      centroid: similarity between the centers of gravity
      average-linkage: average similarity of all cross cluster pairs
  - top-down / **divisive clustering**: groups are split using various criteria
- **partition algorithms**: construct a partition of N objects into a set of K clusters
  - **hard assignment**: K means algorithm (heuristic method)
    - input: $\mathbf{x}^{(i)}, K$
      initialize: the K (random) cluster centers
    - iterate:
      1. assign points to the nearest cluster centers based on some metric
      2. re-estimate the K cluster centers (centroid or mean)
    - terminate: if none of the objects change membership; otherwise go to 1
    - optimization: NP-hard

$$\mathbf{c}, \mathbf{z} = \arg\min_{\mathbf{c},\mathbf{z}} \sum_{i=1}^{N} ||\mathbf{x}^{(i)} - \mathbf{c}_{z^{(i)}}||_2^2$$

      alternating minimization - **block coordinate descent**:
      1. $\mathbf{z} = \arg\min_{\mathbf{z}} \sum_{i=1}^{N} ||\mathbf{x}^{(i)} - \mathbf{c}_{\mathbf{z}^{(i)}}||_2^2$
      2. $\mathbf{c} = \arg\min_{\mathbf{c}} \sum_{i=1}^{N} ||\mathbf{x}^{(i)} - \mathbf{c}_{\mathbf{z}^{(i)}}||_2^2$
    - issues:
      - converge to sub-optimal clustering: seed choice $\rightarrow$ try multiple starting points / choose centers that are far apart
      - assumes isotropic, equal variance, convex clusters
      - sensitive to outliers $\rightarrow$ K-medoids (more work)
      - number of clusters K: look for "knee" in objective function
  - **soft assignment**: mixture modeling (generative approach: probability that an object belongs to a cluster) $\Rightarrow$ **Gaussian Mixture Model (GMM)**
    - **mixture of Gaussians**:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k \boldsymbol{\Sigma}_k)$$

      Each Gaussian density $\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ is called a **component** of the mixture and the parameter π_k is the **mixing coefficient** such that $\forall i, \pi_k \in [0,1], \sum_{k=1}^{k} \pi_k = 1$
    - assuming $p(\mathbf{x}|y=i) \sim \mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$,

$$p(\mathbf{x}) = \sum_{i=1}^{K} p(\mathbf{x}|y=1)p(y=i) = \sum_{i=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$$

$$p(\mathbf{X}) = \prod_j \sum_i p(\mathbf{x}_j|y=i)p(y=i) = \prod_j \sum_i \frac{1}{\sqrt{\boldsymbol{\Sigma}_i}} \exp\left(-\frac{1}{2}(\mathbf{x}_j - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i (\mathbf{x}_j - \boldsymbol{\mu}_i)\right) p(y=i)$$

* **expectation-maximization (EM) algorithm**: given a Gaussian mixture model, the goal is to maximize the likelihood function with respect to the parameters $\boldsymbol{\theta} = \{\pi_i, \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i\}_{i=1}^K$
  - E step: for a fixed set of Gaussian mixture model parameters, $\boldsymbol{\theta}^t$, update the probability that each point, $\mathbf{x}_i$, belongs to cluster k:

$$p(y = k|\mathbf{x}_i, \boldsymbol{\theta}^t) = \frac{p(\mathbf{x}_i|y = k, \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)p(y = k)}{\sum_{j=1}^K p(\mathbf{x}_i|y = j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)p(y = j)} = \frac{\pi_k^{(t)}\mathcal{N}\left(\mathbf{x}_i; \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}\right)}{\sum_{j=1}^K \pi_j^{(t)}\mathcal{N}\left(\mathbf{x}_i; \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)}\right)}, \forall i, k$$

  - M step: re-estimate the parameters $\boldsymbol{\theta}^{(t+1)}$ using the current probabilities:

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_{i=1}^N p\left(y = k|\mathbf{x}_i, \boldsymbol{\theta}^{(t)}\mathbf{x}_i\right)}{\sum_{i=1}^N p\left(y = k|\mathbf{x}_i, \boldsymbol{\theta}^{(t)}\right)}, \forall k$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_{i=1}^N p\left(y = 1|\mathbf{x}_i, \boldsymbol{\theta}^{(t)}\right)\left(\mathbf{x}_i - \boldsymbol{\mu}_i^{(t+1)}\right)\left(\mathbf{x}_i - \boldsymbol{\mu}_i^{(t+1)}\right)^T}{\sum_{i=1}^N p\left(y = 1|\mathbf{x}_i, \boldsymbol{\theta}^{(t)}\right)}, \forall k$$

$$\pi_i^{(t+1)} = \frac{\sum_{i=1}^N p\left(y = k|\mathbf{x}_i, \boldsymbol{\theta}^{(t)}\right)}{N}, \forall k$$

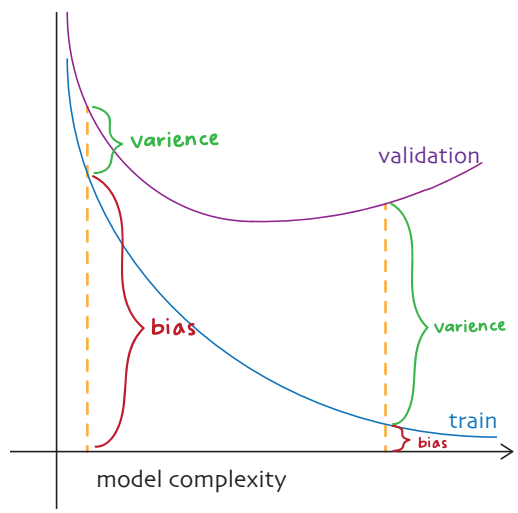
  EM performs coordinate ascent, which can get stuck in local optima

Learning theory

- optimal classification function: $h^* : \mathcal{X} \to \mathcal{Y}$ such that h^* minimizes the rist $R(h)$, the expected loss for a randomly drawn test data (X, Y):

$$h^* = \arg\min_h R(h) = \arg\min_h \mathbb{E}_{XY}[L(Y, h(X))]$$

- weighted loss shifts the decision boundary
- as model complexity increases, bias decreases, and variance increases



- true/generalization error / **expected risk**: $R(h) = p_{\mathbf{x}\sim p^*(\mathbf{x})}(c^*(\mathbf{x}) \neq h(\mathbf{x}))$, which is always unknown
  train error / **empirical risk**: $\hat{R}(h) = p_{\mathbf{x}\sim\mathcal{S}}(c^*(\mathbf{x}) \neq h(\mathbf{x}))$ where $\mathcal{S}$ is the training set
- **PAC (Probably Approximately Correct)/SLT (Statistical Learning Theory) model**:
  1. generate instances from unknown distribution: $\mathbf{x}^{(i)} \sim p^*(\mathbf{x}), \forall i$
  2. oracle labels each instance with unknown function: $y^{(i)} = c^*\left(\mathbf{x}^{(i)}\right), \forall i$
  3. learning algorithm chooses hypothesis $h \in \mathcal{H}$ with the lowest training error: $\hat{h} = \arg\min_{h\in\mathcal{H}} \hat{R}(h)$

4. our goal is to choose hypothesis with lowest generalization error: $h^* = \arg\min_{h \in \mathcal{H}} R(h)$

- **true function** labels training data and is what we're trying to learn: c^*
  expected risk minimizer has the lowest true error: h^*
  empirical risk minimizer has the lowest training error: $\hat{h}$
  c^*, h^* are not necessarily equal

- **PAC learning**:
  - **PAC criterion**: the learner produces a high accuracy learner with high probability

$$p(\forall h, |R(h) - \hat{R}(h)| \leq \epsilon) \geq 1 - \delta$$

  - **sample complexity** is the minimum number of training examples N s.t. the PAC criterion is satisfied.
    If N is finite, then $\mathcal{H}$ is **learnable**. If N is a polynomial function of $\frac{1}{\epsilon}, \frac{1}{\delta}$, then $\mathcal{H}$ is **PAC learnable**.
  - the algorithm is **consistent** if $\forall \epsilon, \delta, \exists N > 0$ s.t. $\forall p^*, p(|R(h) - \hat{R}(h)| > \epsilon) < \delta$
  - **realizable**: $c^* \in \mathcal{H}$
    agnostic: $c^* \in \mathcal{H}$ or $c^* \notin \mathcal{H}$
  - **Theorem 1** - realizable, finite $|\mathcal{H}|$: $N \geq \frac{1}{\epsilon}\left[\ln(|\mathcal{H}|) + \ln\left(\frac{1}{\delta}\right)\right]$labeled examples are sufficient so that with probability $(1-\delta)$ all $h \in \mathcal{H}$ with $\hat{R}(h) = 0$ have $R(h) \leq \epsilon$.

    Proof. Assume k bad hypotheses $h_1, \ldots, h_k$ with $R(h_i) > \epsilon$. Fix h_i,

$$p(h_i \text{ consistent with the first training example}) \leq 1 - \epsilon$$
$$p(h_i \text{ consistent with the first } N \text{ training example}) \leq (1-\epsilon)^N$$

    so p(at least one bad hypothesis h_i is consistent with the first N training example) $\leq k(1-\epsilon)^N \leq |\mathcal{H}|(1-\epsilon)^N$ by union bound.
    Fact: $1 - x \leq \exp(-x) \Rightarrow |\mathcal{H}|(1-\epsilon)^N \leq |\mathcal{H}|\exp(-\epsilon N) \leq \delta \Rightarrow \ln(|\mathcal{H}|) + \ln\left(\frac{1}{\epsilon}\right) \leq \epsilon N$, which gives statement 1: $\underline{N \geq \frac{1}{\epsilon}\ln(|\mathcal{H}|) + \ln\left(\frac{1}{\epsilon}\right)}$.
    Assume statement 1, then with probability $\delta, \exists h \in \mathcal{H}, R(h) > \epsilon, \hat{R}(h) = 0$ and with probability $1-\delta, \forall h \in \mathcal{H}$ with $R(h) > \epsilon$ have $\hat{R}(h) > 0 \Rightarrow \forall h \in \mathcal{H}$ with $R(h) \leq \epsilon$ have $\hat{R}(h) = 0$ by contrapositive. □

    Remarks. bound is <u>inversely linear in ϵ</u> (halving the error requires double the examples) and <u>logarithmic in $|\mathcal{H}|$</u> (quadrupling the hypothesis space only requires double the examples)

  - **Corollary 1** - realizable, finite $|\mathcal{H}|$: for some $\delta > 0$, with probability at least $(1-\delta)$, for any $h \in \mathcal{H}$ with $\hat{R}(h) = 0$,

$$R(h) \leq \frac{1}{N}\left[\ln(|\mathcal{H}|) + \ln\left(\frac{1}{\delta}\right)\right]$$

  - **Theorem 2** - agnostic, finite $|\mathcal{H}|$: $N \geq \frac{1}{2\epsilon^2}\left[\ln(|\mathcal{H}|) + \ln\left(\frac{2}{\delta}\right)\right]$ labeled examples are sufficient so that with probability $(1-\delta)$ for all $h \in \mathcal{H}$ we have that $|R(h) - \hat{R}(h)| \leq \epsilon$.

    Remarks. bound is <u>inversely quadratic in ϵ</u> (halving the error requires 4x the examples) and is <u>logarithmic in $|\mathcal{H}|$</u>

  - **Corollary 2** - agnostic, finite $|\mathcal{H}|$: for some $\delta > 0$, with probability at least $(1-\delta)$, for all hypotheses $h \in \mathcal{H}$,

$$R(h) \leq \hat{R}(h) + \sqrt{\frac{1}{2N}\left[\ln(|\mathcal{H}|) + \ln\left(\frac{2}{\delta}\right)\right]}$$

  - Summary of PAC bounds for finite model classes: with probability $\geq (1-\delta)$,
    * $\forall h \in \mathcal{H}, \hat{R}(h) = 0$, **(Haussler's bound)**

$$R(h) \leq \epsilon = \frac{\ln(|\mathcal{H}|) + \ln\left(\frac{1}{\delta}\right)}{N}$$

* $\forall h \in \mathcal{H}$, **(Hoeffding's bound)**

$$|R(h) - \hat{R}(h)| \leq \epsilon = \sqrt{\frac{\ln(|\mathcal{H}|) + \ln\left(\frac{2}{\delta}\right)}{2N}}$$

- **bias-variance tradeoff**: fix N,

Model class	$\hat{R}(h)$	$R(h)$
$\lvert\mathcal{H}\rvert$ large (complex)	small	large
$\lvert\mathcal{H}\rvert$ small (simple)	large	small

- **shattering**: a set of points S is **shattered** by $\mathcal{H}$ if there're hypotheses in $\mathcal{H}$ that split S in all $2^{|S|}$ possible ways; i.e. all possible ways of classifying points in S are achievable using concepts in $\mathcal{H}$.
- **VC (Vapnik-Chervonenkis) dimension** of a hypothesis space $\mathcal{H}$ is the cardinality of the largest set S that can be shattered by $\mathcal{H}$.
  Fact: if $\mathcal{H}$ is finite, then $\mathsf{VC}(\mathcal{H}) \leq \ln(|\mathcal{H}|)$
  Fact: VC of linear separator in $\mathbb{R}^d$ is $d+1$
  Prove VC requires: $\exists$ a dataset of size d that can be shattered and $\forall$ datasets of size $d+1$ cannot be shattered
  Prove shattering requires $\forall$ labeling of the dataset, the hypothesis class contains a hypothesis that classifies it
- **Theorem 3** - realizable, infinite $|\mathcal{H}|$: $N = O\left(\frac{1}{\epsilon}\left[\mathsf{VC}(\mathcal{H})\ln\left(\frac{1}{\epsilon}\right) + \ln\left(\frac{1}{\delta}\right)\right]\right)$ labeled Examples are sufficient so that with probability $(1-\delta)$ all $h \in \mathcal{H}$ with $\hat{R}(h) = 0$ have $R(h) < \epsilon$.
- **Corollary 3** - realizable, infinite $|\mathcal{H}|$: for some $\delta > 0$, with probability at least $(1-\delta)$, for any $h \in \mathcal{H}$ with $\hat{R}(h) = 0$,

$$R(h) \leq O\left(\frac{1}{N}\left[\mathsf{VC}(\mathcal{H})\ln\left(\frac{N}{\mathsf{VC}(\mathcal{H})}\right) + \ln\left(\frac{1}{\delta}\right)\right]\right)$$

- **Theorem 4** - agnostic, infinite $|\mathcal{H}|$: $N = O\left(\frac{1}{\epsilon^2}\left[\mathsf{VC}(\mathcal{H}) + \ln\left(\frac{1}{\delta}\right)\right]\right)$ labeled examples are sufficient so that with probability $(1-\delta)$ all $h \in \mathcal{H}$ we have $|R(h) - \hat{R}(h)| \leq \epsilon$.
- **Corollary 4** - agnostic, infinite $|\mathcal{H}|$: for some $\delta > 0$, with probability at least $(1-\delta)$, for all hypotheses $h \in \mathcal{H}$,

$$R(h) \leq \hat{R}(h) + O\left(\sqrt{\frac{1}{N}\left[\mathsf{VC}(\mathcal{H}) + \ln\left(\frac{1}{\delta}\right)\right]}\right)$$